

Why Math Beats AI/ML: The Zantoras Approach to Cyber Resilience

The Problem with AI/ML in Cybersecurity

The cybersecurity industry has spent a decade betting on machine learning. Train a model on known threats, feed it data, and hope it generalizes. This approach has a fundamental flaw: **it only finds what it's been taught to look for.**

CyberSense, the industry's leading ransomware detection engine, maintains a database of 7,500+ ransomware variants built from 120 million+ samples. It uses 200+ content-inspection metrics trained on known attack patterns. This is impressive engineering -- and it is structurally fragile.

When a novel ransomware variant encrypts your backup data using a method that doesn't match any of those 7,500 known patterns, the model is extrapolating beyond its training distribution. It produces a probability score. And that score is only as reliable as the statistical similarity between the new threat and whatever the model was trained on.

Zantoras takes a different approach. We use math.

Math is Deterministic. AI/ML is Probabilistic.

Property	Mathematical Detection	AI/ML Detection
Output	Deterministic measurement	Probability estimate
Zero-day detection	Yes -- measures shape change	Limited -- generalizes only within training distribution
Explainability	"Entropy increased 4.2 standard deviations"	"Model confidence: 87%"
Court admissibility	Reproducible, verifiable	Black box, challenged under Daubert
Training data required	None -- learns baselines from live data	Supervised: large labeled datasets; unsupervised: unlabeled corpora; both require representative data
Model drift	Baselines adapt continuously via EMA	Requires periodic retraining on new data
Adversarial resistance	Attacker must manipulate physical properties of data	Models can be fooled by adversarial inputs
Day-one readiness	Works immediately from baseline	Requires training or tuning period before production accuracy
Computational cost	Lightweight arithmetic	GPU-intensive inference
Inspectability	Transparent box -- every intermediate value visible and auditable	Black box -- weights and activations are opaque

How Math Detects What AI/ML Cannot

The Evidence Engine deploys 17 mathematical detectors organized into four families:

- **Statistical** -- Z-Score, CUSUM, Mahalanobis Distance, EMA, Hurst Exponent
- **Information-Theoretic** -- Shannon Entropy, Entropy Rate, Compression Ratio, Kolmogorov Complexity (NCD)
- **Structural / Distributional** -- Benford's Law, Zipf's Law, Fuzzy Pattern Hashing, Cosine Similarity, Jensen-Shannon Divergence
- **Temporal / Frequency-Domain** -- Cadence Detection, Spectral FFT, Wavelet Decomposition

Each family attacks the problem from a mathematically independent angle. An adversary who defeats one family still faces three others.

1. Shannon Entropy -- Encryption Has a Shape

Encrypted data has near-maximum entropy (~8.0 bits/byte). Normal backup data does not. This is not a learned pattern -- it is an information-theoretic fact.

When ransomware encrypts files, the entropy of those files shifts measurably. No signature database needed. No training required. The math is the same whether the ransomware was written yesterday or ten years ago.

Detection: If backup data entropy spikes from a baseline of 5.2 to 7.9 bits/byte, something encrypted it.

Handling partial encryption: Some ransomware encrypts only file headers (first 4KB) to evade full-file entropy analysis. This is why entropy is one detector in a composite pipeline -- partial encryption still disrupts compression ratios, changes delta patterns, and alters behavioral cadence. No single detector is sufficient. The composite catches what any individual method might miss.

2. Z-Score -- Statistical Deviation is Universal

A z-score measures how many standard deviations a value is from the mean. The Evidence Engine applies z-scores to:

- **Bytes per flow** -- sudden volume changes
- **Packet count** -- traffic pattern shifts
- **Flow duration** -- connection timing changes
- **Backup change rate** -- data mutation velocity

At a threshold of 3.0 standard deviations, the false positive rate is 0.3%. This is not a tuned parameter from a training set -- it is a property of the normal distribution.

Detection: If tonight's backup change rate is 4.1 standard deviations above the 30-day baseline, something abnormal happened. The math doesn't care what caused it.

3. Cosine Similarity -- Behavioral Shape Comparison

The Evidence Engine represents each entity's behavior as a feature vector (bytes, packets, duration) and compares it against its baseline using cosine similarity -- the angle between two vectors in n-dimensional space.

- **1.0** = identical behavioral shape
- **0.0** = completely different behavior

This captures proportional changes that absolute thresholds miss. A server that normally sends 1GB in 1000 packets over 60 seconds looks different from one sending 10GB in 10000 packets over 600 seconds -- but the *shape* is identical (cosine similarity ≈ 1.0). A server that suddenly sends 1GB in 10 packets over 2 seconds has the same volume but a completely different shape.

Detection: Ransomware changes the shape of data flow. The math measures the angle change.

A note on AI/ML: Cosine similarity is used inside many AI/ML models. The difference is transparency. In an AI/ML pipeline, cosine similarity is one operation buried inside hidden layers -- the output is a probability with no visible reasoning. In the Evidence Engine, the cosine similarity score is the output: visible, auditable, and reproducible by any third party.

4. Cadence Detection -- Timing Has a Rhythm

Normal systems produce predictable timing patterns. Backups run on schedules. Logs arrive at regular intervals. The Evidence Engine tracks:

- **Average interval** between events (Welford's algorithm for streaming variance)
- **Burst detection** -- sudden clusters of activity
- **Silence detection** -- unexpected gaps

A ransomware attack disrupts timing cadence before it disrupts data content. The math catches the rhythm change.

Detection: If a backup system that writes every 300+/-15 seconds suddenly writes 47 times in 10 seconds, the cadence detector fires -- before any content analysis is needed.

5. Compression Ratio -- Redundancy is Measurable

Normal data compresses predictably. Encrypted data does not compress at all (it's already maximum entropy). The Evidence Engine tracks compression ratios via delta compression with XOR differencing.

- **Normal backup delta:** 3:1 to 10:1 compression ratio (redundant, compressible)
- **Encrypted backup delta:** ~1:1 compression ratio (random, incompressible)

Detection: When the compression ratio of backup deltas drops from 5:1 to 1.02:1, the data has been encrypted. No signatures needed.

6. Fuzzy Pattern Hashing -- Structure, Not Content

The Evidence Engine normalizes log entries by replacing variable content (IPs, timestamps, UUIDs, numbers) with tokens, then hashes the structural skeleton. Logs with 80%+ structural similarity cluster together automatically.

This creates behavioral fingerprints without AI/ML classification. When a new log type appears or existing patterns shift, the math detects it as a structural anomaly.

Detection: A server that normally produces 12 log pattern types suddenly producing 47 pattern types indicates compromise -- measured by clustering math, not AI/ML classification.

7. Exponential Moving Average -- Trend Without Training

Risk scores are smoothed using exponential moving average ($\alpha=0.1$): 10% weight on new observations, 90% on historical baseline. This eliminates single-event noise while catching sustained trends.

Unlike AI/ML models that need retraining when behavior shifts, EMA adapts continuously and transparently. The formula is auditable. The weights are visible. There is no hidden layer.

8. CUSUM -- Catching Slow Drift That Z-Scores Miss

Cumulative Sum (Page, 1954) is the mathematically optimal detector for small persistent shifts in a mean. Where a z-score asks "is this single observation extreme?", CUSUM asks "has the process been consistently drifting?"

The Evidence Engine runs two-sided CUSUM on bytes, packets, and duration per source. It accumulates

deviations over time. A slow data exfiltration that adds 2% more bytes per flow -- invisible to z-scores -- will trigger CUSUM after enough samples accumulate.

Detection: An attacker slowly exfiltrating data at 2% above normal per flow goes unnoticed by z-score thresholds. CUSUM accumulates the deviation: after 50 flows, the cumulative sum crosses the alarm threshold. Math catches the drift.

9. Mahalanobis Distance -- Correlated Anomalies

Z-scores treat each metric independently. Mahalanobis distance accounts for correlations between them. A server running 80% CPU with 90% memory is normal (they correlate). The same server at 80% CPU with 10% memory is suspicious -- the correlation is broken.

The Evidence Engine tracks a full 3x3 covariance matrix (bytes, packets, duration) using Welford's online algorithm, inverts it via Cramer's rule, and computes $D^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$. No external math libraries. Pure Go arithmetic.

Detection: A compromised server producing high bytes but unusually low packets (large payloads, few connections -- data staging) scores normal on individual z-scores but anomalous on Mahalanobis because the metric correlation is broken.

10. Benford's Law -- Nature Has a Digit Distribution

The first digits of naturally occurring numbers follow $P(d) = \log_{10}(1 + 1/d)$. Digit 1 appears 30.1% of the time. Digit 9 appears 4.6%. This is not a learned rule -- it is a mathematical constant derived from scale invariance.

Packet sizes, byte counts, flow durations, and transaction amounts in normal enterprise traffic all follow Benford's Law. Synthetic data, fabricated logs, and attack-generated traffic violate it.

Detection: The Evidence Engine accumulates leading digits and runs a chi-squared goodness-of-fit test against the Benford distribution. If DGA-generated domain lookups produce uniformly distributed byte counts, the chi-squared statistic spikes. No training set. The expected distribution is a mathematical constant.

11. Zipf's Law -- Distribution Shape Has a Signature

In natural datasets, the k-th most frequent item appears with frequency proportional to $1/k^s$. Normal DNS queries, log categories, and API calls follow a steep Zipf distribution (exponent $s \approx 1.0-2.0$) -- a few items dominate, most are rare.

Attacks flatten this distribution. DGA domains, brute-force login attempts, and network reconnaissance spread activity across many targets equally, dropping the Zipf exponent below 0.5.

Detection: The Evidence Engine fits the Zipf exponent via log-log linear regression. An R^2 near 1.0 means the distribution is healthy. A flattened exponent with low R^2 means the natural power-law structure has been disrupted.

12. Spectral Analysis (FFT) -- Finding Hidden Frequencies

The Evidence Engine runs a full Cooley-Tukey radix-2 FFT on time-series data, transforming from time domain to frequency domain. Normal enterprise traffic has natural periodicities: 24-hour cycles, weekly patterns, maintenance windows. C2 beaconing introduces new, artificial frequencies.

Spectral flatness measures how concentrated energy is across frequencies. Low flatness (energy concentrated in few bins) with a dominant peak indicates periodic behavior -- a mathematical signature of C2 beaconing.

Detection: A compromised host beaconing to a C2 server every 60 seconds creates a sharp spectral peak at $1/60$

Hz. No signature database needed. The FFT reveals the hidden rhythm. Pure math, implemented in Go without external libraries.

13. Entropy Rate -- Sequential Randomness, Not Just Static

Shannon entropy measures the randomness of a single distribution. Entropy rate measures the randomness of a process over time -- how predictable is the next state given the current one?

Normal protocols are structured state machines with low entropy rate (the next packet type is predictable from the current one). Encrypted tunnels, covert channels, and compromised processes have high entropy rate -- every transition is unpredictable.

Detection: The Evidence Engine builds a Markov transition matrix from observed state sequences, computes the stationary distribution, and calculates $h = -\sum p_i \log p_i$. A DNS-tunneling attack that encodes data in query strings produces near-maximum entropy rate. The math detects the hidden channel.

14. Wavelet Decomposition (Haar DWT) -- Anomalies at Every Timescale

Different anomalies manifest at different timescales. A packet storm is a seconds-scale event. A service disruption is minutes-scale. Gradual capacity exhaustion is hours-scale. A single detection window catches only one scale.

The Evidence Engine performs Haar wavelet decomposition, splitting signals into multiple resolution levels simultaneously. Energy analysis at each scale reveals where anomalies live. Donoho's universal threshold ($T = \sigma \sqrt{2 \log n}$) -- derived from MAD-based robust noise estimation -- separates signal from noise without any training.

Detection: A DDoS ramp-up shows low energy at fine scales (individual packets look normal) but high energy at coarse scales (the aggregate trend is anomalous). Wavelet decomposition catches it at the right timescale. Z-scores and entropy, operating at a single scale, miss it.

15. Kolmogorov Complexity (NCD) -- Algorithmic Incompressibility

Shannon entropy measures statistical randomness. Kolmogorov complexity measures algorithmic randomness -- the length of the shortest program that can reproduce the data. Encrypted, obfuscated, or AI-generated data has near-maximal complexity (incompressible). Normal operational data has lower complexity (repeating structure, compressible).

The Evidence Engine approximates Kolmogorov complexity via Normalized Compression Distance (NCD) using DEFLATE. It tracks the normalized complexity of data chunks over time and computes NCD between the current window and the historical baseline: $NCD(x,y) = (C(xy) - \min(C(x), C(y))) / \max(C(x), C(y))$, where $C()$ is compressed size.

Detection: A backup volume that normally has complexity 0.35 (structured, compressible) suddenly shows complexity 0.97 (incompressible). The NCD between current and baseline spikes to 0.9. The data has been encrypted or replaced with synthetic content -- detected without signatures, without training, using only the mathematical properties of compression.

16. Jensen-Shannon Divergence -- Distribution Drift Detection

Benford's Law and Zipf's Law test against specific theoretical distributions. Jensen-Shannon Divergence (JSD) generalizes this to compare any two empirical distributions -- packet size histograms, byte value distributions, inter-arrival time profiles, or API call frequency spectra.

JSD is symmetric and bounded: $JSD(P||Q) = 0.5 KL(P||M) + 0.5 KL(Q||M)$, where $M = 0.5(P+Q)$ is the mixture distribution. Range: 0.0 (identical) to 1.0 (maximally different). Unlike raw KL divergence, JSD is always finite and

never requires both distributions to share the same support.

Detection: An attacker subtly shifts the packet size distribution -- slightly larger payloads, slightly different frequency profile -- to evade threshold-based detectors. JSD catches the distribution drift even when no individual value crosses any threshold. The math measures the shape of the entire distribution, not just its outliers.

17. Hurst Exponent -- Long-Range Memory Disruption

The Hurst exponent (H) measures the long-term memory of a time series via Rescaled Range (R/S) analysis. Normal system behavior exhibits persistent long-range dependence ($H > 0.5$) -- today's traffic correlates with yesterday's. Random noise has $H \sim 0.5$. Anti-persistent (choppy, erratic) behavior has $H < 0.5$.

The Evidence Engine computes H over sliding windows using R/S analysis at multiple segment sizes, fits the exponent via log-log regression, and tracks shifts in H over time. A sudden drop in H indicates that the system's natural temporal "memory" has been disrupted.

Detection: A file server that normally shows persistent write patterns ($H \sim 0.72$) suddenly drops to $H \sim 0.41$ during a ransomware encryption event -- the natural long-range correlation of backup writes is replaced by choppy, anti-persistent noise. The Hurst exponent catches the moment the system's rhythm breaks, even before volume or entropy change.

Why This Matters for Backup Recovery

The CyberSense Approach (AI/ML)

1. Scan backup data against 7,500+ known ransomware signatures
2. Run 200+ content-inspection metrics trained on 120M+ samples
3. Produce a confidence score
4. If confidence exceeds threshold, flag as corrupted

Failure mode: Novel ransomware that doesn't match known patterns produces a low confidence score. The backup is declared "clean" when it is not.

The Zantoras Approach (Math)

1. Measure entropy of backup data -- is it encrypted?
2. Measure compression ratio of backup deltas -- is redundancy gone?
3. Measure z-score deviation of backup change rate -- is mutation velocity abnormal?
4. Measure cosine similarity of behavioral shape -- has the pattern changed?
5. Measure cadence deviation -- has timing rhythm broken?
6. Measure CUSUM drift -- is there a slow, persistent shift that individual checks miss?
7. Measure Mahalanobis distance -- are metric correlations broken?
8. Test Benford's Law -- do the digit distributions match natural data?
9. Fit Zipf exponent -- has the frequency distribution flattened?
10. Run FFT spectral analysis -- are there hidden periodicities (beaconing)?
11. Compute entropy rate -- are state transitions becoming unpredictable?
12. Decompose via wavelets -- are anomalies hiding at specific timescales?

13. Measure Kolmogorov complexity -- has the data become algorithmically incompressible?
14. Compute Jensen-Shannon divergence -- has the distribution shape shifted?
15. Estimate Hurst exponent -- has the system's long-range memory been disrupted?
16. Chain all measurements into a tamper-evident hash chain -- can anyone prove the results were altered?

Failure mode: An attacker would need to encrypt data without changing its entropy or algorithmic complexity, compress it without changing compression ratios, mutate it without changing the change rate, alter flow patterns without changing cosine similarity, maintain natural digit distributions, preserve Zipf power-law structure, keep distribution histograms unchanged, avoid introducing new frequencies, keep state transitions predictable, maintain long-range temporal correlations, and stay invisible across all wavelet timescales -- simultaneously. Evading one detector is plausible. Evading 17 independent mathematical detectors drawn from four orthogonal families -- simultaneously, without leaving a trace in any dimension -- is computationally and operationally infeasible in a way that fooling a single AI/ML classifier is not.

Distinguishing Attacks from Legitimate Changes

A common question: how does math-based detection distinguish ransomware encryption from an administrator enabling BitLocker, or a user compressing files?

The answer is **context correlation across multiple detectors**:

- **Legitimate encryption** (BitLocker rollout) is preceded by change tickets, happens during maintenance windows, follows cadence patterns, and is performed by authorized processes. The cadence detector sees scheduled activity. The z-score sees expected volume changes.
- **Ransomware encryption** happens outside maintenance windows, disrupts timing cadence, produces abnormal process patterns, and mutates data at rates that deviate from baseline. Multiple detectors fire simultaneously.

A single entropy spike is not an alert. A simultaneous entropy spike + cadence disruption + compression ratio collapse + z-score deviation across multiple metrics is. The composite pipeline requires convergence across independent mathematical dimensions before flagging an anomaly.

Addressing false positives: Alert fatigue is the silent killer of security operations. AI/ML classifiers tune a single confidence threshold and accept the resulting false-positive rate. The Evidence Engine uses the composite as a *consensus engine* -- an alert fires only when multiple independent detectors agree. Because each detector measures a different physical property of the data, correlated false positives across all of them are statistically rare. The result: high sensitivity to real attacks with a structurally low false-positive rate, without requiring threshold tuning on labeled attack data.

The Evidence Gap

Detection is only half the problem. After an attack, organizations need to **prove** their recovery is clean. This is where AI/ML completely fails:

Requirement	AI/ML	Math
Reproducibility	Difficult to reproduce -- model versions, weights, and inference paths vary	Run the formula again -- identical result every time
Audit trail	Model weights are opaque	Hash chain is independently verifiable

Requirement	AI/ML	Math
Court testimony	"The model said 87% confident" -- challenged under Daubert for explainability	"Entropy was 7.94 bits/byte, 4.2sigma above baseline" -- reproducible, verifiable
Insurance proof	Self-attestation of tool output	Cryptographic proof with tamper-evident chain
Regulatory compliance	Vendor-dependent certification	NIST SP 800-53 CP-9 specifies cryptographic hashes
Third-party verification	Requires same model, same version, same weights	Anyone with SHA-256 can verify

44% of cyber insurance claims were denied in 2024. The leading cause: inability to prove controls were operational during the incident window. AI/ML tools produce outputs. Math produces evidence.

Composite Detection Pipeline

The Evidence Engine doesn't rely on any single method. It fuses all mathematical detectors using weighted composite scoring:

Real-Time Per-Record Detectors

Detector	Weight	What It Measures
Cosine Similarity	25%	Behavioral shape deviation
Mahalanobis Distance	20%	Correlated multivariate anomalies
Z-Score	15%	Statistical outlier magnitude
CUSUM	15%	Slow persistent drift detection
Cadence	15%	Timing rhythm disruption
Destination Frequency	10%	Connection rarity

Each detector produces a score from 0.0 (anomalous) to 1.0 (normal). The weighted composite provides a single, explainable risk score that combines 6 independent mathematical perspectives on the same data -- in real time, per record.

Windowed Batch Analyzers

Analyzer	What It Measures
Benford's Law	Digit distribution anomalies (fraud, synthetic data)
Zipf's Law	Frequency distribution flattening (DGA, brute-force)
Spectral / FFT	Hidden periodicities and C2 beaconing
Entropy Rate	Sequential randomness (covert channels, tunneling)
Wavelet DWT	Multi-timescale anomalies (ramp-up attacks, slow degradation)

Analyzer	What It Measures
Kolmogorov / NCD	Algorithmic incompressibility (encryption, synthetic/AI-generated data)
Jensen-Shannon Divergence	Arbitrary distribution drift (subtle payload changes)
Hurst Exponent	Long-range temporal memory disruption (behavioral regime change)

Windowed analyzers operate over batches of records. They detect structural patterns that no single record can reveal -- frequency domain signatures, distribution shapes, multi-scale energy concentrations, and long-range temporal correlations.

Why these weights: Cosine similarity (25%) captures the broadest behavioral shape change. Mahalanobis (20%) catches correlated anomalies that independent z-scores miss. Z-score (15%) and CUSUM (15%) provide complementary statistical views -- z-score for single-point outliers, CUSUM for accumulated drift. Cadence (15%) detects timing disruptions that precede data corruption. Destination frequency (10%) supplements with connection rarity. The eight batch analyzers provide independent structural validation. Critically, detectors span four orthogonal families (statistical, information-theoretic, structural, temporal) so that no single evasion technique can suppress them all. Weights are configurable per deployment and can be tuned to specific environments without retraining a model.

Performance: Every detector is deterministic arithmetic -- addition, multiplication, division, logarithms. No GPU required. No model inference latency. The full 6-detector composite runs at line speed on commodity hardware; windowed batch analyzers process on configurable intervals. Total overhead is a fraction of what a single AI/ML inference pass costs.

No black box. No hidden layers. No training data. **A transparent box: 17 mathematical detectors, zero AI/ML dependencies, every intermediate value inspectable.**

Summary

AI/ML tools are powerful for pattern recognition in domains with stable distributions and abundant labeled data. Cybersecurity has neither. Threats evolve daily. Novel attacks are the ones that matter most. And when the attack is over, you need proof that stands up in court -- not a probability score from a model no one can inspect.

The Evidence Engine runs 17 mathematical detectors -- Shannon entropy, z-scores, cosine similarity, cadence analysis, compression ratios, fuzzy pattern hashing, exponential moving averages, CUSUM, Mahalanobis distance, Benford's Law, Zipf's Law, spectral FFT analysis, Markov entropy rate, Haar wavelet decomposition, Kolmogorov complexity via NCD, Jensen-Shannon divergence, and Hurst exponent -- all implemented in pure Go with zero AI/ML dependencies, chained into a tamper-evident cryptographic audit trail. Every measurement is deterministic. Every result is reproducible. Every proof is independently verifiable.

CyberSense needs 7,500 known ransomware signatures to detect 7,500 known threats. The Evidence Engine needs zero signatures to detect any threat that changes the shape of data -- including the 7,501st variant that hasn't been written yet.

AI/ML predicts. Math proves.